# OOP Assignment: 02

**Assignment 1:** A bank has an ATM account with an initial balance of Tk 10,000. Three customers attempt to withdraw Tk 4,000, Tk 3,000, and Tk 5,000 simultaneously. Develop a Java program using OOP concepts to create a BankAccount class with a withdraw() method and create a separate thread for each customer. Each thread should attempt to withdraw money from the same account and display whether the transaction was successful or failed due to insufficient funds. Display the final account balance and observe how the execution order of the threads affects the output.

**Assignment 2**: Hospital Management System A hospital needs to display patient information and update appointment records simultaneously. Create two threads in Java to perform these operations concurrently using the Thread class or Runnable interface.

**Assignment 3:** Two customers attempt to book seats on a train that has only one seat remaining. Create multiple threads to simulate the booking process and use synchronization to prevent the same seat from being booked twice.

**Assignment 4:** A railway station has two employees: one sells train tickets, while the other displays available seat information. Create a Java program using multithreading where one thread books a ticket and updates the available seat count, while another thread displays the remaining seats simultaneously. Use the Thread class or Runnable interface to create the threads and observe how both threads execute concurrently.

**Assignment 5:** A railway station has only one seat remaining, and two ticket-booking employees attempt to sell that seat to different passengers simultaneously. Create a Java program using multithreading without synchronization to simulate the booking process. Observe the output and explain how a race condition may occur and why synchronization is necessary to prevent the same seat from being booked twice.